

Security & Preferences Hardening Report

SportsBackend · 2026-05-20

1. Executive Summary

The SportsBackend application underwent a focused security audit and a targeted feature expansion in the same pass. Seven security hardening changes were applied across authentication, error handling, CORS, and session management. In parallel, seven new fields were added to the UserPreference entity to support richer home-page recommendations (time-of-day, weekday/weekend availability, play intensity, intent, push-notification radius, and last-active timestamp). The Maven build completed with BUILD SUCCESS after all changes. No existing controller endpoint signatures changed, so the modifications are backward-compatible for the frontend.

2. Files Created

- bullet exception/NotFoundException.java
- bullet exception/UnauthorizedException.java
- bullet exception/ForbiddenException.java
- bullet exception/BadRequestException.java
- bullet exception/GlobalExceptionHandler.java
- bullet model/TimeOfDay.java
- bullet model/Intensity.java
- bullet model/LookingFor.java

3. Files Modified

- bullet security/SecurityConfig.java
- bullet service/JwtUtil.java
- bullet controller/UserController.java
- bullet dto/ChangePasswordRequest.java
- bullet model/UserPreference.java
- bullet resources/application.yml

4. Security Changes

4.1 Global exception handler

Introduced a `@RestControllerAdvice` class, `GlobalExceptionHandler`, that catches every uncaught exception and returns a safe JSON body. Previously, throw new `RuntimeException(...)` statements scattered throughout the service layer surfaced as opaque HTTP 500s with default Spring error pages that could leak class names, message text, and stack traces.

Exception types now mapped to specific HTTP statuses:

- bullet NotFoundException → 404
- bullet UnauthorizedException → 401
- bullet ForbiddenException → 403

- bullet BadRequestException → 400
- bullet AccessDeniedException, AuthenticationException (Spring Security) → 403 / 401
- bullet MethodArgumentNotValidException, ConstraintViolationException → 400
- bullet MissingServletRequestParameterException, MethodArgumentTypeMismatchException → 400
- bullet DataIntegrityViolationException → 409 (no SQL details returned to client)
- bullet IllegalArgumentException → 400
- bullet Catch-all Exception → 500 with generic body; full stack trace logged at ERROR.

Impact: Mitigates information-disclosure (CWE-209). Stack traces, ORM messages, and internal class names no longer reach the client.

4.2 CORS lockdown

The previous configuration used `setAllowedOrigins(Arrays.asList("*"))`, which permits cross-origin requests from any domain. In conjunction with credentialed requests this would allow a malicious page in a user's browser to read responses from the API on their behalf.

Replaced with:

- bullet Origins read from `app.cors.allowed-origins`, sourced from env (`APP_CORS_ALLOWED_ORIGINS`).
- bullet Defaults to local dev origins only (`localhost:3000`, `localhost:8081`, `localhost:19006`).
- bullet If the configured value contains `"*"`, the application throws at startup — no silent fallback to wildcard.
- bullet Allowed headers restricted to `Authorization`, `Content-Type`, `Accept`, `X-Requested-With` (instead of all headers).
- bullet `allowCredentials` now `true`, valid because the wildcard is removed.

4.3 Stateless session policy

Added `sessionManagement(SessionCreationPolicy.STATELESS)`. The application previously issued `JSESSIONID` cookies on every request even though authentication is JWT-only. Stateless mode eliminates server-side session storage and removes the attack surface for session fixation and cookie-based replay.

4.4 Hardened security response headers

Spring Security now emits:

- bullet `X-Frame-Options: DENY` — mitigates clickjacking by preventing the API from being embedded in iframes.
- bullet `X-Content-Type-Options: nosniff` — mitigates MIME-sniffing attacks on uploaded or returned content.
- bullet `X-XSS-Protection: 0` — explicit per modern guidance (legacy headers replaced by CSP in browsers that support it).
- bullet `Cache-Control: no-cache, no-store, must-revalidate` — prevents intermediate caches from retaining authenticated responses.

4.5 JWT secret validated at boot

`JwtUtil` now uses a `@PostConstruct` initializer to verify the signing key at application startup:

- bullet Refuses to start if `jwt.secret` is missing or blank.

- bullet Refuses to start if the value is not valid Base64.
- bullet Refuses to start if the decoded key is shorter than 32 bytes (256 bits), the minimum for HS256.
- bullet Refuses to start if `jwt.expiration` is non-positive.
- bullet Caches the parsed key — previous code decoded the secret on every call.
- bullet `validateToken` now catches `JwtException` and returns `false` for malformed tokens instead of bubbling a 500.
- bullet Password encoder upgraded from default BCrypt cost (10) to **cost 12** (~4x slower hashing, materially harder to brute-force).

4.6 Application error response leak prevention

Added to `application.yml`:

```
server:
  error:
    include-stacktrace: never
    include-message: never
    include-binding-errors: never
    include-exception: false
```

These settings prevent Spring's default error response body from containing exception messages, binding details, exception type, or stack traces. The `GlobalExceptionHandler` produces all client-facing error bodies instead.

4.7 UserController hardening

- bullet Removed `throw new RuntimeException("Invalid Token")` from `getUserIdFromToken`. Replaced with `UnauthorizedException`, which now returns a proper 401 instead of a misleading 500.
- bullet Added `@Valid` to the `changePassword` request body so the new constraints on `ChangePasswordRequest` (non-blank, 8–128 character new password) actually fire.
- bullet Added `@Validated` to the controller class so `@Size(max=100)` on the search query parameter is enforced.
- bullet Defensive length cap of 100 characters on the search query inside the controller body, even if Bean Validation is bypassed somehow.

5. Why SQL Injection Is Not A Concern

All custom queries in the repository layer use named parameters (`:name`) bound through Spring Data's `@Param`. These parameters are passed to JDBC via `PreparedStatement.setObject(...)` — the value never participates in SQL string construction, so the underlying driver escapes and types it correctly. A user supplying `' ; DROP TABLE venues ; --` as a parameter would be searched for literally, not interpreted as SQL.

Spring Data method-name queries (e.g. `findByOwner_Id(UUID id)`) translate to JPQL, which similarly uses parameter binding. Safe by construction.

Two ways to **re-introduce** SQL injection (do not do these):

- bullet Building SQL with `String.format` or string concatenation: `"SELECT * FROM users WHERE email = '" + email + "'"`.

but `EntityManager.createNativeQuery("... WHERE col = " + userInput)` — same problem.

If a query genuinely needs dynamic structure (sortable columns, optional filters), use Spring Data Specification, QueryDSL, or carefully validated allow-lists for column names. Never concatenate untrusted strings into SQL.

6. Preference Field Additions (Home-Page Recommendations)

Seven columns were added to the `user_preferences` table via fields on `UserPreference`. Each enables a distinct filter on the home-page recommendation feed.

Column	Type	Purpose / Example query
<code>preferred_time_of_day</code>	enum	MORNING/AFTERNOON/EVENING/ANYTIME. Filter <i>“players who play in the evening”</i> .
<code>available_weekdays</code>	boolean	Match weekday-only players together; default <code>true</code> .
<code>available_weekends</code>	boolean	Match weekend warriors together; default <code>true</code> .
<code>preferred_intensity</code>	enum	CASUAL/MODERATE/COMPETITIVE. Separate “just for fun” from tournament-level players.
<code>looking_for</code>	enum	PARTNER/TEAMMATES/LESSONS/SOCIAL/ANY. Surface coaches separately from match-seekers.
<code>notification_radius_meters</code>	integer	Per-user push-alert radius for “new match nearby” notifications.
<code>last_active_at</code>	timestamp	Rank recently-active users higher in matchmaking results.

Three new enum types support these columns: `TimeOfDay`, `Intensity`, `LookingFor`. All are stored as `VARCHAR` via `@Enumerated(EnumType.STRING)` for forward compatibility (renaming an enum constant requires only a code change plus an `UPDATE` statement, not a schema migration).

7. Build Verification

After all changes were applied, the project was compiled with:

```
./mvnw -B compile
```

The Maven output ended with `[INFO] BUILD SUCCESS`. No compilation errors, no missing imports, no broken references. The hardened code is ready to rebuild and deploy.

8. Recommended Next Steps

Deploy the new image: `docker compose up -d --build app`.

Restart the app at least once after deployment so the JWT key validator runs — a misconfigured secret will now fail fast.

- bullet Before any production deploy, set `APP_CORS_ALLOWED_ORIGINS` to the actual frontend origin(s). The default values cover local dev only.
- bullet Sweep remaining `throw new RuntimeException(...)` calls in `VenueService` and similar service classes; replace each with the most fitting typed exception (`NotFoundException`, `BadRequestException`, etc.).
- bullet Introduce Flyway (or Liquibase) for schema management. The new preference columns will currently be created by Hibernate `ddl-auto: update`, which is acceptable in dev but unsafe for prod.
- bullet Add a rate limiter (Bucket4j, Resilience4j, or a reverse-proxy rule) on `/api/auth/login` and `/api/auth/register` to make credential-stuffing attacks impractical.
- bullet Add Testcontainers integration tests for the spatial repository methods so regressions surface in CI instead of in production.

9. What Was Not Changed

- bullet Repository / SQL queries — already safe via parameter binding.
- bullet Existing controller endpoint signatures — no breaking changes for the frontend.
- bullet Database schema for existing tables — only additive changes (new columns are nullable except the two booleans, which default to `true`).
- bullet Production secrets handling — `application-secrets.properties` is already gitignored; env-var-driven config is already in place.
- bullet Business logic in services — behavior is unchanged; only error reporting and validation were touched.

End of report.